

Finding Uncertainty in DynaMix

A master practical report for the Koppe group in WS 2026

Birk Källberg – April 2026

github.com/Fjallriipa/DynaMix

1 Introduction

1.1 DynaMix

DynaMix is a Dynamical System Reconstruction (DSR) foundation model by the Durstewitz group that attempts to reconstruct the complete Dynamical System (DS) from only sparse observational data. [Dynamix] It's trained on a synthetic dataset of diverse, simulated dynamical systems. Several 'expert' ALRNNs - in of themselves a ML model used for DSR [ALRNNs] - are trained and used in next-step prediction while a Gating network uses a context - the sparse observational data - to output a weighting of the experts that reflects the dynamical regime of the current prediction step. This setup achieves zero-shot DSR on a small fraction of compute of similarly capable DSR models [Dynamix].

1.2 Goal of this practical

However, DynaMix is a largely deterministic model, meaning there are no obvious ways to check how successful a given reconstruction is without access to a dense ground truth. Which we don't have in a typical DynaMix use case.

Also, as a more out there hope, it would be really cool if one could use DynaMix for active learning techniques, where the model tells us what additional measurements to do in order to get a fuller picture of the system. Such techniques fall under the umbrella of Optimal Experiment Design (OED) but they universally require some sort of statistical model of both the system we're studying and of how additional measurements help pin down the parameters of that statistical model. [OED] With DynaMix, we usually lack both.

In this practical, my goal was to find some way to reliably estimate DynaMix's uncertainty or expected prediction error. If successful, DynaMix reconstructions become a lot more real-world informative and further applications like active learning can be developed.

1.3 DynaMix's architecture

Normal DSR can be done with ALRNNs. However, these can be only trained to one specific system at a time. DynaMix has a whole set of 'expert' ALRNNs and uses for its next-step prediction a weighted sum of their outputs. The weighting (which experts to

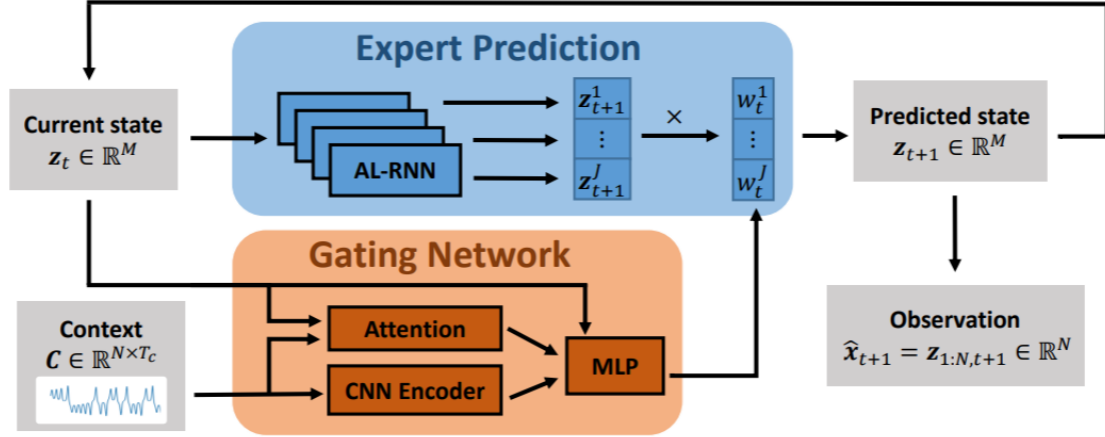


Figure 2: Illustration of the DynaMix architecture. At test time, only a context signal C is provided and guides the selection of experts to yield arbitrarily long forward predictions of the dynamics.

Figure 1: Architecture illustration from [Dynamix]

consult at any point in time) is computed by the Gating Network (GN). The experts are trained along with the GN and their individual behavior is not legible. On their own, they all produce unstable systems, which are only reigned in by the careful weighting of the GN into the context’s chaotic attractor.

The Gating Network works essentially by checking what parts of the context data are most relevant for the current state. I.e. what data points come closest to the current position. That info, together with the current state gets used as input to an MLP to determine the expert weights. My interpretation of what this setup does internally, is that the GN seems to look at what the local dynamics of the context data are to choose a combination of experts that will minimize expected distance of the prediction to the ground truth at the next time step.

That is also how DynaMix is trained: MSE Loss over the prediction-to-ground-truth-distance together with sparse teacher forcing [STF].

2 Methodology

2.1 The short-term forecasting dataset

As mentioned, DynaMix is largely deterministic. However, there is one source of random noise it uses in each prediction step. In the Gating Network, where the previous prediction (the latent state) is converted into a position so that the closest context data points can be found, that latest position gets a roughly 5% gaussian noise ϵ added, which results in a bit noisy prediction of the next step. This noise source is there because it turned out to stabilize the training process, which would otherwise too easily collapse into k-cycles [true? -j, ask andre]. A kind of regularization parameter so to say. Even at inference time, it tends to lead to more even coverage of the chaotic attractor than without. [opt. show img. in Appendix.]

As DynaMix can be trained in batch parallel, so can its predictions be run as a batch. Due to the noise source ϵ , this results in an empirical distribution of where DynaMix

Example of the short-term forecasting dataset

Track 1/1000, $T = 0 \dots 100$

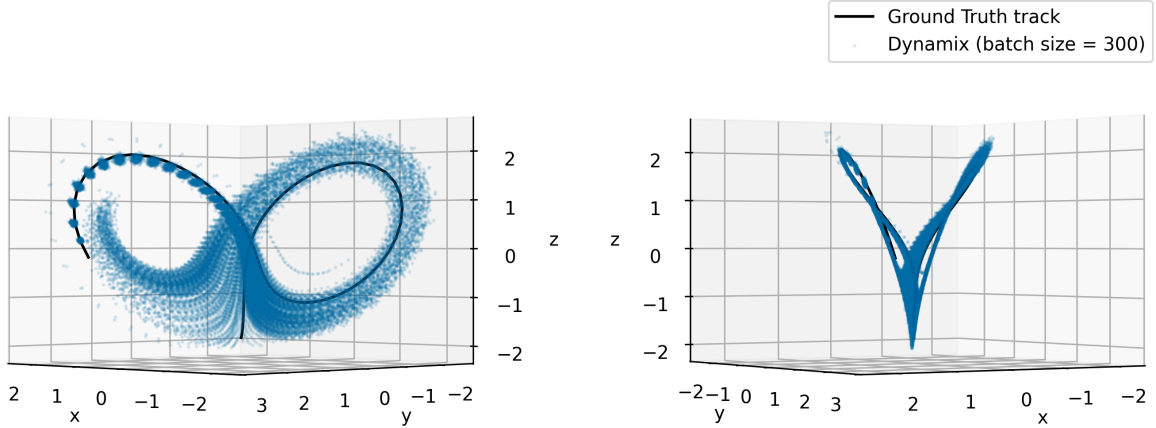


Figure 2: example track

All 1000 starting points
of the short-term forecasting dataset

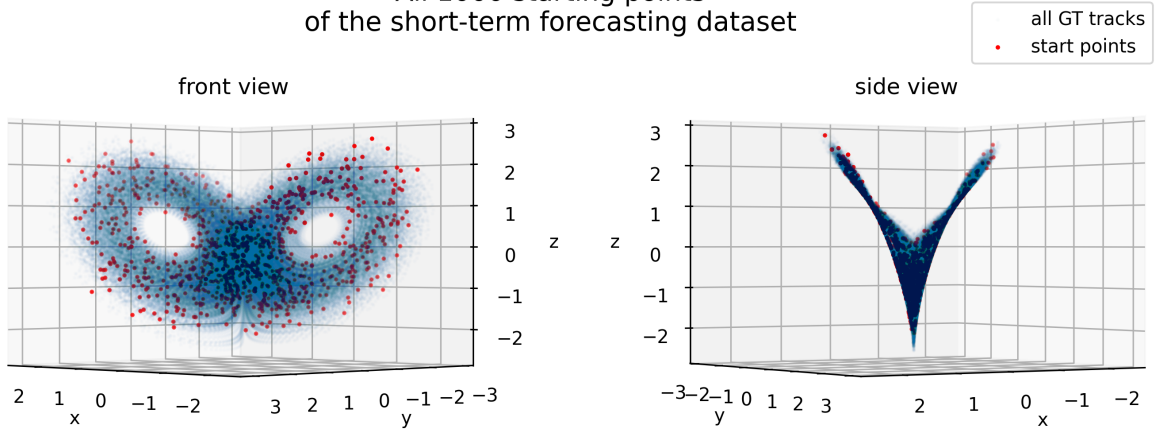


Figure 3: track starting points

predictions end up. In this practical, I focused mostly on trying to find out if that distribution also is a good uncertainty measure or if it's fundamentally uncalibrated and we have to find sth. else.

To do that, I built quite a large synthetic dataset of a ground truth simulation and corresponding DynaMix predictions on the Lorenz63 system. [Lorenz] For simplicity, in all my experiments, I worked only with that system. For implications of that, see the [Discussion section].

The dataset contains a very large ground truth: 200,000 time steps of the simulated Lorenz63 attractor, using the same simulation library [dysts] that created the DynaMix synthetic training set. This ground truth is split into 1,000 200-time-step long pieces, which I call *tracks*. For each track, I created a batch of 300 DynaMix predictions. These predictions use the first point of the track as their starting point and a separate 10k-time-

Example of Ground Truth Uncertainty

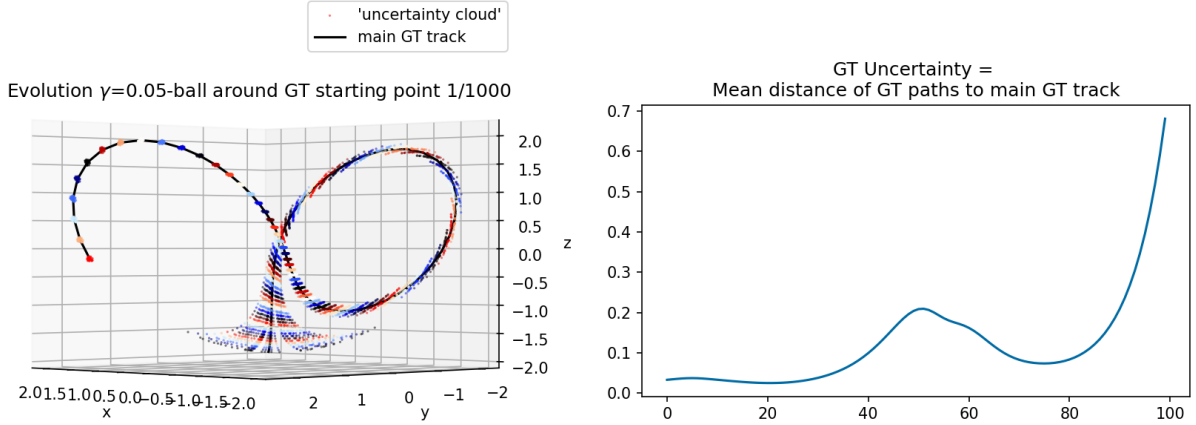


Figure 4

step context I had chosen earlier. [opt. reference Discussion of Dynamix fragility] The idea was to create lots of short-term prediction + ground truth pairs all over the attractor and then be able to compare those two. As can be seen in the example [see img.], Dynamix prediction quickly diverge from their ground truth tracks. For readability, only the first 100 time steps are shown as the Lyapunov time of the Lorenz63 system is 110 time steps in this simulation. The other plot [see img.] shows how the track starting points are distributed evenly across the whole attractor, enabling the dataset to completely cover all dynamic regimes of the attractor.

2.2 Finding a good uncertainty measure in DynaMix

I tried out lots of ways to measure DynaMix uncertainty and its deviation from ground truth and even tried to find ways to measure ground truth uncertainty itself. What worked out best in the end was the following: The DynaMix batch prediction cloud starts out as something like a gaussian ball but soon smears out into something shapeless. [see. dataset ex.] So trying to measure the volume of the gauss ball would not do. Instead, mean distance of each point in the cloud to the cloud's center proved to be a much more robust measure. I'll just call this measure "DynaMix Uncertainty" from here on.

The ground truth Uncertainty, I discovered, can be computed without additional costly simulations. The simulated 200,000-time-step Lorenz attractor is already quite dense. One can isolate all the different paths within a $\gamma=0.05$ -ball around a starting point, and plot their time evolution. This way one has a 'probability' cloud similar to the DynaMix cloud which also diverges over time because this is a chaotic system. Ground Truth Uncertainty then is an analogous uncertainty measure to the DynaMix one. [see GT Unc. img.]

These are the three measures I ended up with. The question now becomes: Do they correlate?

- DynaMix Uncertainty: $\text{mean}_{batch}(\|\hat{X}(t) - \text{mean}_{batch}(\hat{X}(t))\|_2)$

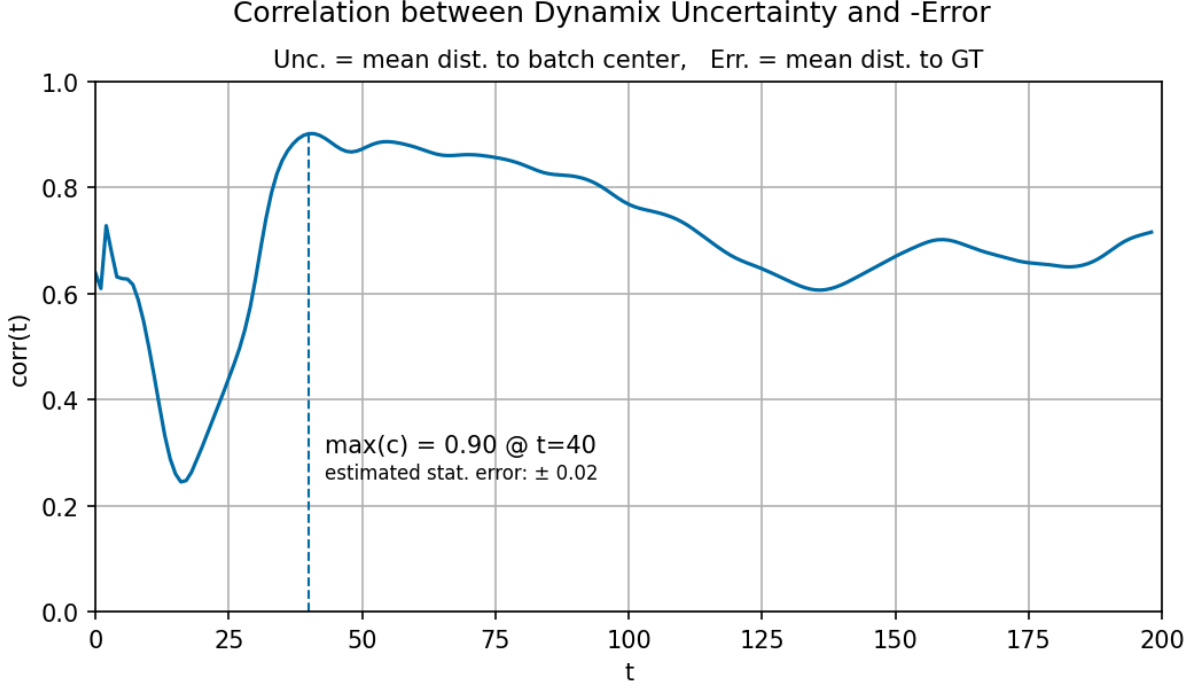


Figure 5

- DynaMix Error: $\text{mean}_{\text{batch}}(\|\hat{X}(t) - X(t)\|_2) = \text{Mean Squared Error}$
- Ground Truth Uncertainty: $\text{mean}_{\gamma\text{-ball}}(\|X_\gamma(t) - X(t)\|_2)$

In theory, a perfect correlation between DynaMix Uncertainty and DynaMix Error would mean that we'd have a ground-truth-less measure of how good our DynaMix prediction is. We could use it on real world data to see how well our short-term prediction worked. A perfect correlation between DynaMix Uncertainty and GT Uncertainty would mean that DynaMix captures the the dynamics of the attractor well, another measure like geometric misalignment D_{stst} [D_stst] indicating that DSR worked.

3 Experimental results

To investigate the correlation between these measures, I created plot showing the their correlation coefficient over all 1,000 tracks and for each of the 200 time steps. [see. img. dyn.err. and gt.unc.]

This does look promising: At least at $t=40$ for Dynamix Error and $t=36$ for GT Uncertainty, there's a ≥ 0.90 correlation to Dynamix Uncertainty. However, I discovered a confounding variable: speed.

So I had to control for that variable by calculation the partial correlation [par.corr.]. For the results, see [img. par.corr. 1 and 2].

As it turns out, at the correlation peaks, the confounding influence of speed is minimal and we're still left with a peak correlation of Dynamix Uncertainty and -Error of 0.88 at $t=40$ and respectively 0.92 at $t=35$ for Dynamix- and GT Uncertainty.

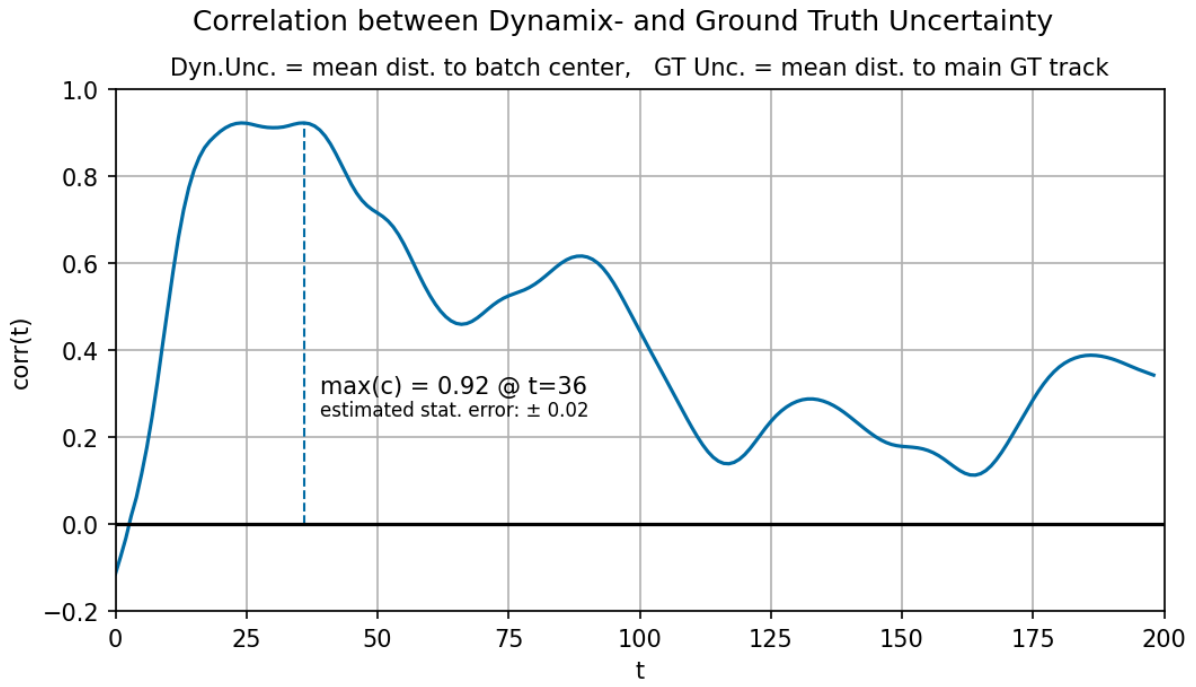


Figure 6

Confounding variable: Speed

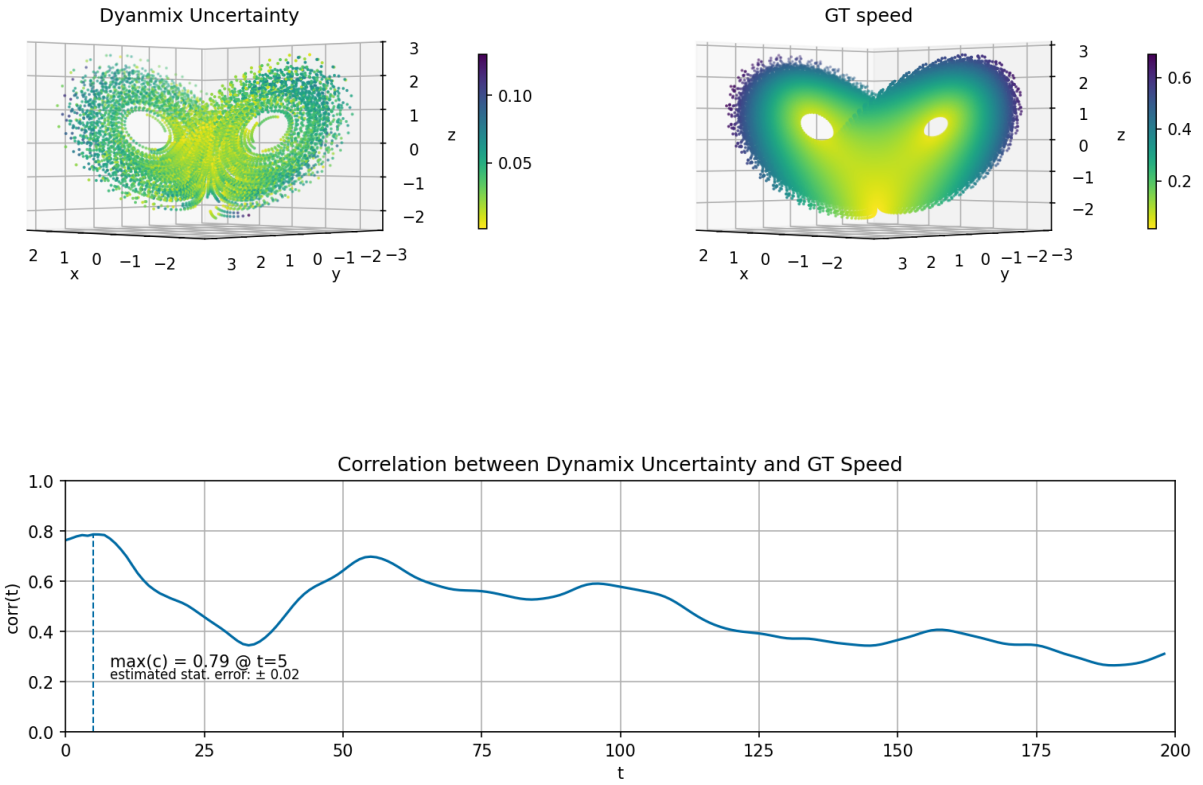


Figure 7

Impact of GT Speed on the correlation between Dynamix Uncertainty and -Error

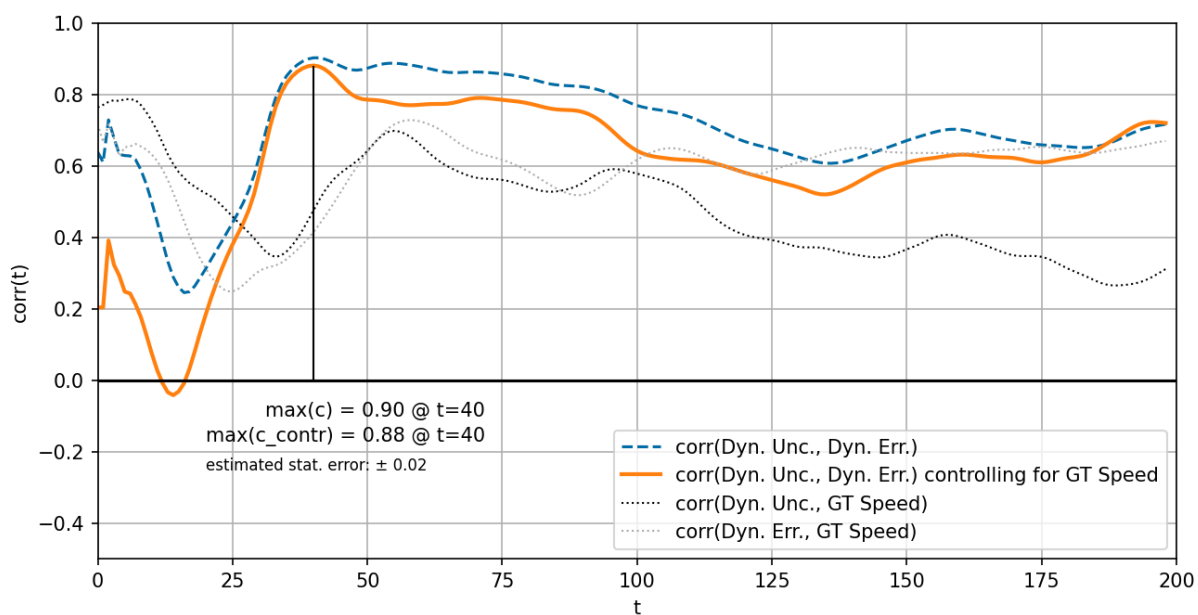


Figure 8

Impact of GT Speed on the correlation between Dynamix- and GT Uncertainty

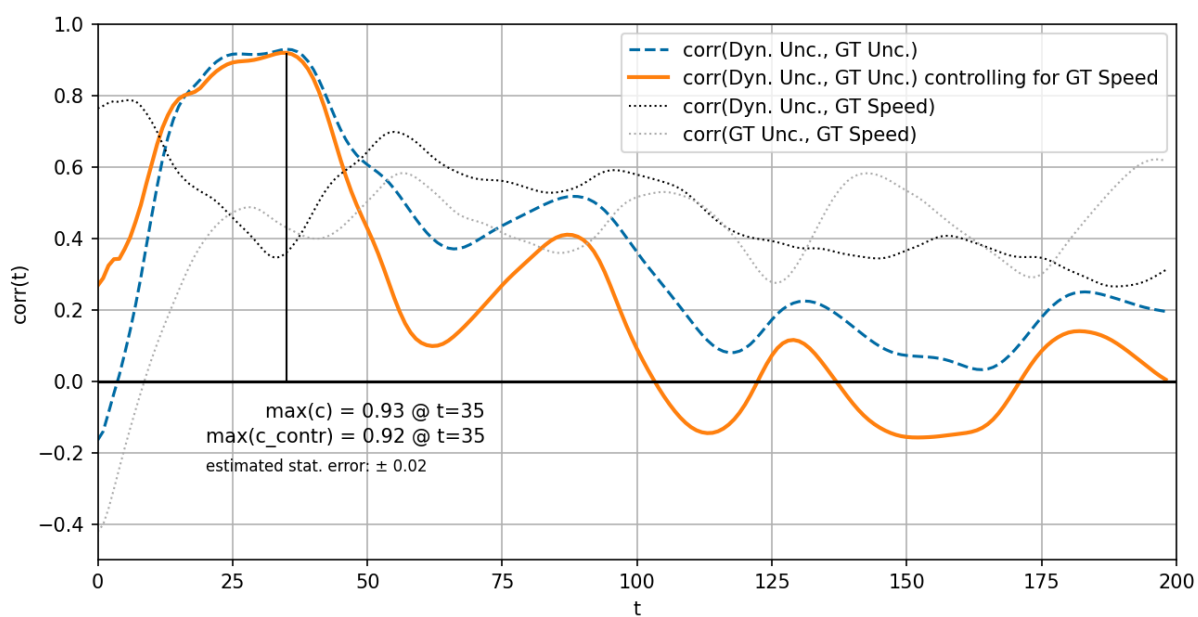


Figure 9

Now, for the application scenario of trying to get an estimate of how reliable the current prediction is, another metric is more helpful. In that scenario, one has only access to the DynaMix uncertainty. So, I calculated a 95% confidence interval of the Dynamix Error and the GT Uncertainty as a factor of the Dynamix uncertainty.

- forecasting error = 1.0x - 4.0x DynaMix uncertainty [95% CI]
- GT uncertainty = 0.13x - 0.91x DynaMix uncertainty [95% CI]

The distributions are not Gaussian and have very thin tails, meaning upper and lower bounds are quite a useful way of expressing these measure estimates. Caveats follow in the next section.

4 Discussion

4.1 What is now possible

What do these results actually tell us? On the one hand, it looks like they give us something useful for cases where we use DynaMix to reconstruct dynamical systems from empirical data. At $t=40$, we seem to be able to pretty confidently estimate the DynaMix forecasting error and the DS uncertainty and thus be able estimate the validity of our reconstruction and properties of the reconstruction just through this new uncertainty measure.

On the other hand, there are some really important caveats to these findings:

4.2 Caveat 1 - The experiments were only done on the Lorenz63 system

It's a) not clear if peak correlations are that strong for other systems. b) It's not clear if the peaks are at the same spot for other systems, or at least a predictably similar spot. c) It's not clear either how well the correlation holds if the reconstruction is better or worse.

There are clear ways to answer these questions, however: a) can be tested by repeating the experiments on other synthetic systems. Same with b). If peaks are not at the same spot, one idea to find the peak spot on novel systems would be to use auto-correlation of DynaMix predictions. Maybe that could reveal the peak location. c) could be tested with different contexts that lead to better or worse reconstructions as measured with the geometric misalignment D_{stst} .

All 3 assumptions need to hold for the measure Dynamix Uncertainty to be an actually useful indicator of short-term prediction accuracy and of long-term reconstruction quality on new empirical data.

4.3 Caveat 2 - This measure can't distinguish between DynaMix's and GT's uncertainty

One would both like to know the system's inherent uncertainty becomes large as well as (and crucially, separately) when DynaMix's prediction or reconstruction gets uncer-

tain because of poor data availability in a particular region of the attractor (or unusual dynamics it hasn't encountered in training before).

The experiment suggested for caveat 1a) (testing correlation for different reconstruction qualities) would answer the question which of these quantities Dynamix Uncertainty actually tracks. If it doesn't track prediction / reconstruction quality, then this measure wouldn't be useful for active learning / OED either and one would need to look for another one.

4.4 Caveat 3 - good correlation only t=40 doesn't seem super useful

With DynaMix Uncertainty as it currently is, we don't know how good our next step prediction is. We only know roughly how good it is 40 steps down the line, where the prediction is a lot more uncertain anyway. It would be much more useful to have a good correlation from t=0 all the way to the Lyapunov time (t=110 in the Lorenz system).

But maybe one could change the training of Dynamix to make its Uncertainty an actually calibrated measure of prediction quality. All it would take would be to add an extra loss term to force it to have good uncertainty correlation with the ground truth at all time steps.

$$\mathcal{L}_{Dyna.Unc.} = \text{DynaMix Error} = \text{mean}_{batch}(\|\hat{X}(t) - X(t)\|_2) = \text{Mean Squared Error}$$

One could even extend this "calibration training" to training data that is made intentionally noisier or more sparse and make DynaMix have accurate / calibrated uncertainty even for those settings with poorer data quality and consequently poorer reconstruction. With luck this would even help with the "mode collapse" tendency that DynaMix has on poorer quality data.

If this training proposal works out as hoped, DynaMix predictions would become be a lot more informative and useful than they are now.

4.5 Summary

In this practical, I found an uncertainty measure for the otherwise quite deterministic DynaMix in a specific setting. But this finding would have to be tested in a lot more conditions to be a useful predictor in a real-world use case. Also I've outlined some ideas for how to modify DynaMix's training to enable a much more robust and useful uncertainty measure than the one I've found so far.

4.6 final remarks

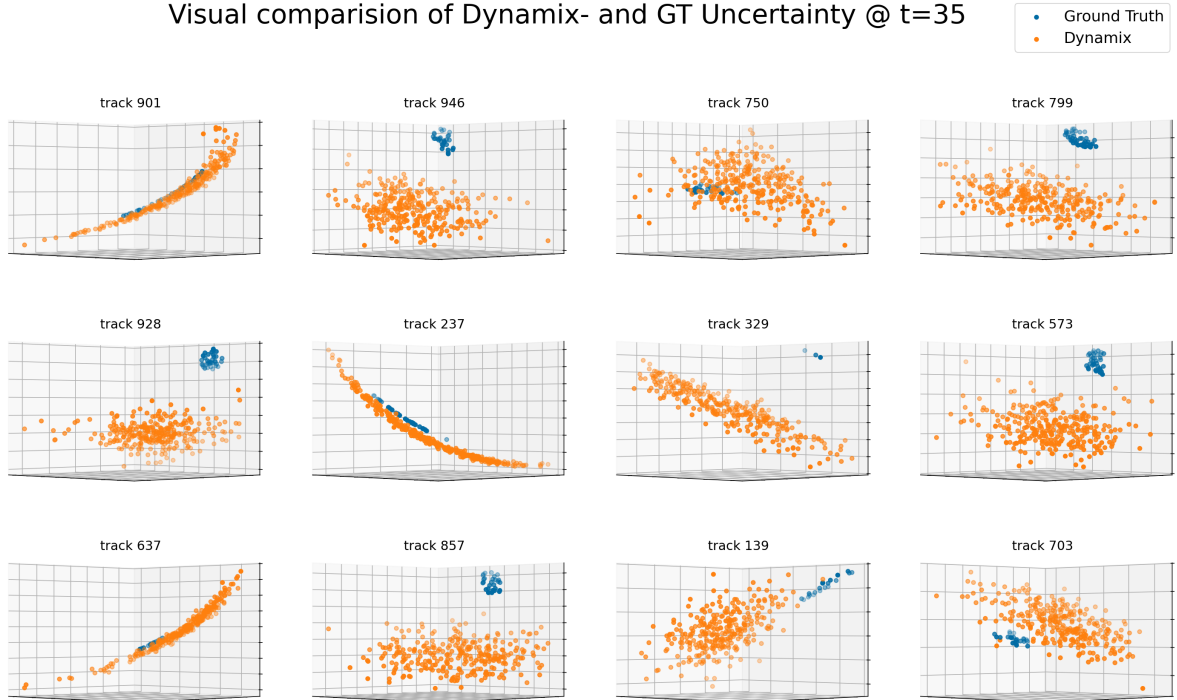
Finally, there are some small side findings I wanted to share that ought to be useful if any of you consider working more with DynaMix: 1. DynaMix is more brittle than I thought

2. I've created an implementation of the DynaMix forecasting function that's 20

3. My code and experiments are on GitHub. [! tidy up code, experiments, explanations first !]

That's it from me. Any questions?

Visual comparison of Dynamix- and GT Uncertainty @ $t=35$



5 Appendix

([Maybe: Show some plots (e.g. 3x3) with Dyn cloud and GT cloud at $t=40$ for different ICs. -j better intuition of what this uncertainty and good correlation looks like / means in practice])